

Summary of NanoFlow: Towards Optimal Large Language Model Serving Throughput

Haoran Jin allenjin@umich.edu ; Yuewen Hou isaachyw@umich.edu; Dhanvi Bharadwaj ghanvib@umich.edu ; Rudrajyoti Roy rudraroy@umich.edu ;

Problem and Motivation

As the demand for LLM serving continues to surge, throughput has emerged as a key performance metric. A significant issue with existing serving engines is their failure to achieve optimal compute utilization. This inefficiency arises because LLM serving consists of heterogeneous operations—compute-bound, memory-bound, and network-bound—that are executed sequentially on a single device. This sequential process creates "pipeline bubbles," periods where the most constrained resource, compute, is left idle. The proposed solution, NanoFlow, addresses this by splitting large input batches into smaller "nano-batches". Since different nano-operations can operate on separate nano-batches without data dependencies, operations that use different resources (e.g., memory and compute) can run in parallel within a single device, maximizing resource utilization.

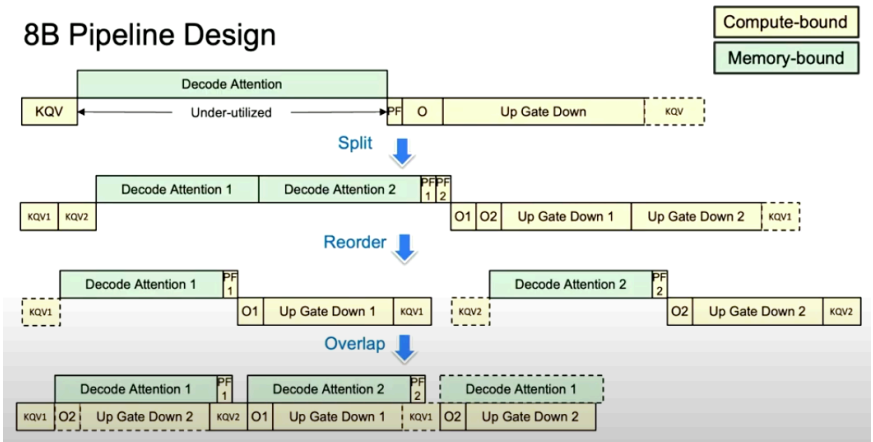
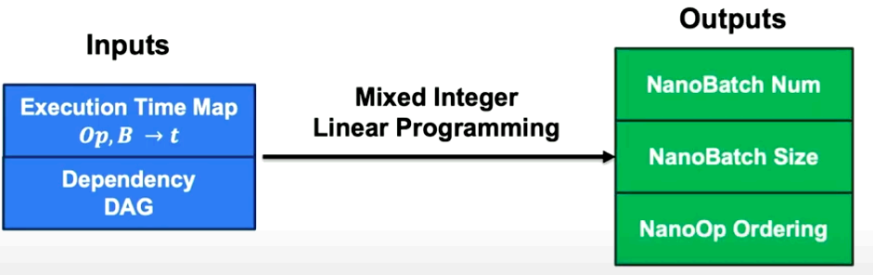
Related Works

- LLM serving optimizations
 - **Orca** , **DistServe** , **Splitwise** , and **DeepSpeed-FastGen** focus on request-level optimizations such as continuous batching and chunked prefilling to combine better prefill and decode phases. However, these approaches do not perform scheduling at the finer granularity of individual operations.
 - **PageAttention**, quantization approaches optimize specific operations. NanoFlow is complementary to these methods and can integrate their optimized kernels into its framework.
- Operational-level parallelism
 - **Rammer** and **Unity**, have explored intra-operation parallelism. While they didn't consider the different resource demands of operations.
 - **ASPEN** and **Welder** break the boundary between operations, but they require significant manual effort and recompilation for new models.

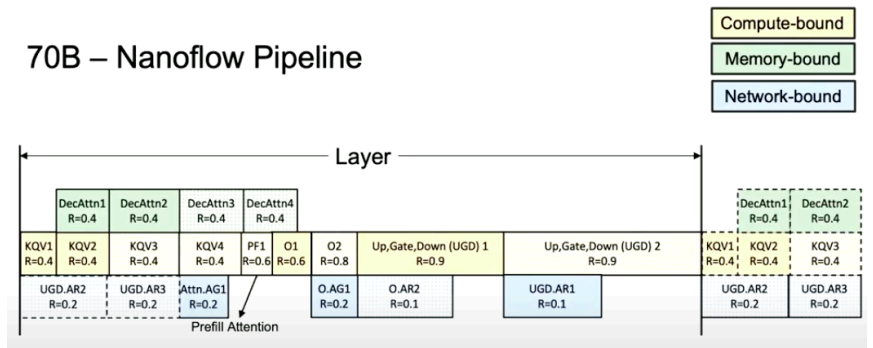
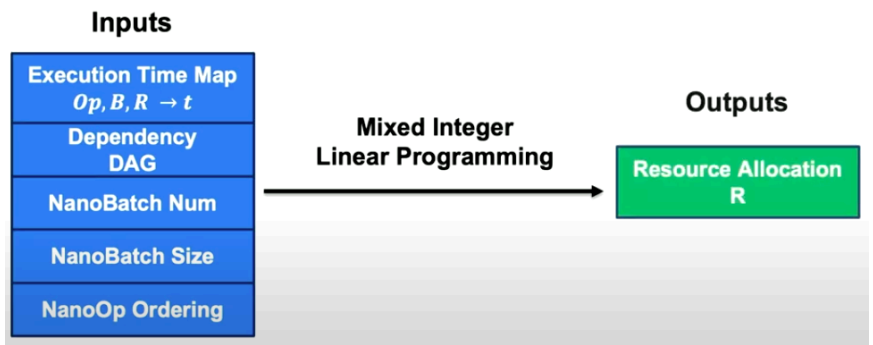
Solution Overview

NanoFlow automates the creation of an optimal execution pipeline using a two-stage search process with mixed integer linear programming techniques and profiling.

Stage I: Determines the pipeline's structure, including the nano-batch number, size, and order of nano-operations, assuming no interference between them



Stage II: Refines the pipeline by profiling and modeling the performance interference between concurrently running kernels, then allocates GPU resources to minimize execution time.



Runtime System: An end-to-end runtime executes the generated pipeline, handling asynchronous request scheduling and KV-cache management.

Limitations

- NanoFlow is primarily optimized for throughput and uses large batch sizes. Consequently, at low request rates, it can exhibit slightly higher latency compared to other systems.
- The auto-search engine simplifies the complex issue of kernel interference by modeling it on a pairwise basis (e.g., compute-memory). This approximation might not fully capture the dynamics when three or more kernel types overlap, potentially leading to a suboptimal pipeline schedule.
- The act of splitting inputs into nano-batches reduces the efficiency of batching effects and can degrade performance if not properly overlapped. An ablation study showed that nano-batching alone, without overlapping, reduces performance by 13.2%.

Future Research Directions

- NanafLOW considers throughput as the main metric when using mixed integer programming to optimize the scheduling. A natural next step is to generalize the model to include factors like the energy consumption of different operations.
- The current auto-search is performed offline for a specific model and workload. Future work could explore dynamically adapting the pipeline in real-time based on fluctuating request patterns.
- Since NanoFlow prioritizes throughput at the cost of slightly higher latency on low loads, a future system could dynamically switch between throughput- and latency-optimized scheduling strategies based on request volume.

Summary of Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot

Problem and Motivation

LLM serving systems face an inherent mismatch between compute-heavy **prefill** stages and memory-heavy **decode** stages. Running both on the same GPU causes contention for resources and reduces throughput. Traditional systems further suffer from **local prefix caching**, where only the GPU that computed a prefix can reuse it. When other GPUs encounter identical prefixes, they must recompute them, wasting valuable compute cycles and inflating latency. Mooncake addresses these inefficiencies by redefining the KVCache as a **global, shareable resource** across all GPUs and servers. The system's guiding principle is to trade inexpensive storage and fast interconnect bandwidth for reduced redundant computation, ensuring high throughput while minimising latency targets.

Related Works

Prior works in LLM serving have primarily focused on targeted scheduling and batching efficiency within single-node or tightly coupled GPU clusters.

vLLM introduced continuous batching, which improved decoding throughput but maintained local cache boundaries.

sarathi-serve explored chunked prefill and stage disaggregation but faced network bottlenecks that limited scalability.

DistServe extended the idea of distributed serving by separating compute across nodes and coordinating data movement between stages, yet it lacked a fully integrated global cache that could efficiently manage prefix sharing at scale.

Also, we covered **Nanoflow** in the last section.

Solution Overview

Mooncake implements a **KVCache-centric disaggregated architecture** that separates computation-heavy prefill work from memory-bound decoding, coordinated by a global scheduler and dynamic cache system.

Key Components:

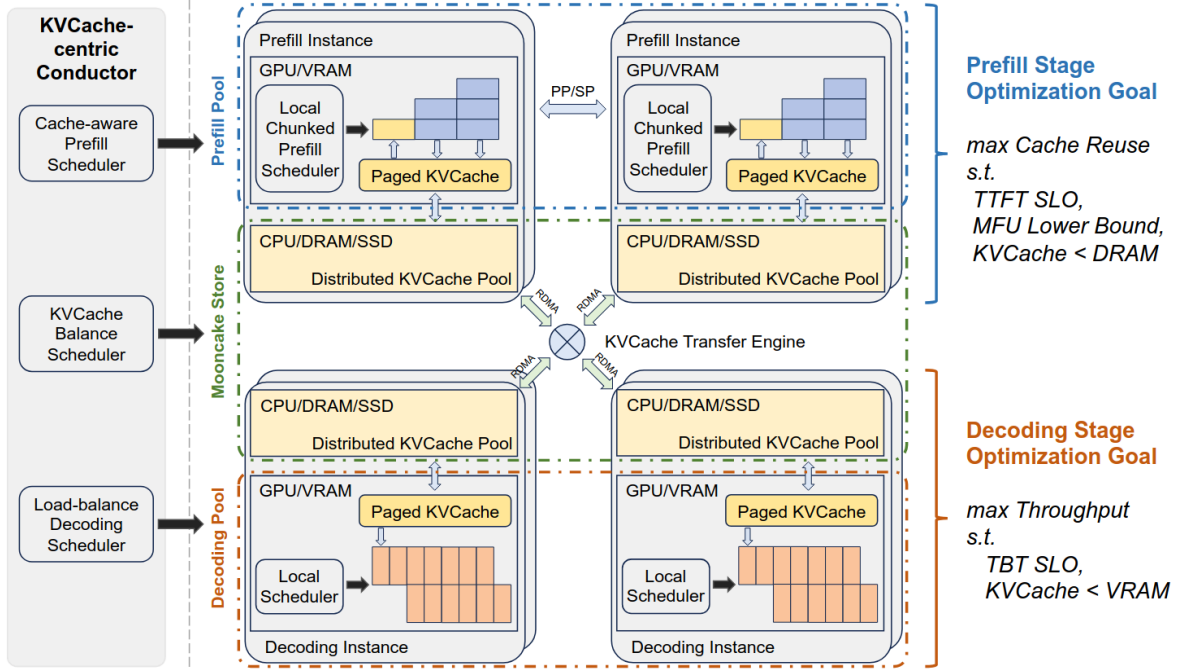


Figure 2: MOONCAKE Architecture.

- **Prefill Pool** – Executes dense compute operations for input processing, generates KVCACHE blocks, and streams partial results.
- **Decode Pool** – Performs autoregressive decoding with continuous batching, using cached KV blocks directly from memory without recomputation.
- **Mooncake Store (Global KVCACHE Layer)** – Distributed cache unifying CPU, DRAM, SSD, and RDMA memory. Stores KVCACHE as paged blocks indexed by prefix hashes for deduplication. Supports zero-copy RDMA transfers, replication, and eviction.
- **Conductor Scheduler** – Assigns prefill and decode nodes for each request. Its objectives are to maximise KV reuse, maintain load balance, and satisfy latency targets for **TTFT** and **TBT** through cache-aware routing and asynchronous transfer coordination.

Inference Workflow

1. **Prefix Reuse:** The **Conductor scheduler** identifies existing cached prefixes in the **Mooncake Store** using prefix hashes and retrieves matching KV blocks via RDMA. Prefill GPUs skip recomputation for these tokens, reducing startup latency.
2. **Incremental Prefill:** The prefill node computes only the

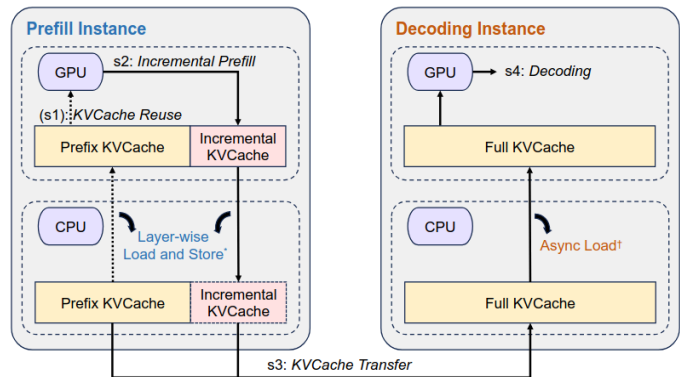


Figure 3: Workflow of inference instances.

uncached portion of the input sequence, producing new KVCache entries in layer-wise chunks and uploading them asynchronously to the global cache.

3. **Async Streaming:** Newly generated KVCache blocks are streamed to decode nodes in parallel with ongoing prefill computation, overlapping communication and computation.
4. **Decoding and Batching:** Decode GPUs merge streamed KVCache into their local buffers and process tokens through **continuous batching**, maintaining high utilisation while meeting per-token latency targets.
5. **Cache Update:** Completed KVCache blocks are written back to the **Mooncake Store** as immutable entries, replicated or evicted as needed for future reuse.

Limitations

- **Network Dependence:** The system relies on high-bandwidth RDMA or NVLink links. If interconnect speed drops below roughly 100 Gbps per GPU, KVCache transfers can take longer than recomputation, eroding performance gains.
- **Scheduler Overhead:** The centralised **Conductor** must track cache placement, reuse, and latency across nodes. This coordination adds complexity and can become a bottleneck during high request loads or when cache locality is uneven.
- **Workload Sensitivity:** Mooncake performs best with overlapping prefixes. In workloads with unique inputs and little reuse, the benefits of caching decline, while transfer and scheduling overheads remain.
- **Hardware Uniformity:** The design assumes homogeneous GPUs and stable inter-node latency. Heterogeneous clusters or uneven link speeds can lead to imbalanced performance and inefficient scheduling.
- **Cache Growth:** Immutable and replicated cache blocks increase memory and SSD consumption. Without careful eviction policies, storage pressure can rise quickly under heavy or bursty workloads.

Future Research Directions

- **Improving the Conductor scheduler** to handle large-scale coordination more efficiently and reduce bottlenecks under heavy load.
- **Supporting heterogeneous hardware** so that Mooncake can operate across GPUs of different types or generations without losing performance.
- **Developing adaptive caching mechanisms** to predict and replicate “hot” prefixes while evicting rarely used ones dynamically.
- **Evaluating energy efficiency and thermal behaviour** to understand Mooncake’s long-term deployment impact and optimise for power use.

Summary of Class Discussion (Q&As)

Q: Why doesn't Nanoflow compare against Sarathi-Serve if it's already using Sarathi-Serve's scheduler?

A: NanoFlow uses Sarathi-serve's batch formation as its baseline, but doesn't directly compare against it. The reported improvement over VLLM ($\sim 4.18\times$) is only slightly higher than sarathi-serve's ($\sim 3.5\text{--}4\times$), so the gain over sarathi-serve itself is small. The real contribution is introducing finer-grained overlap of compute and network, and showing that many inference workloads are compute-bound rather than memory-bound.

Q: Isn't the compute vs memory-bound behavior input dependent?

A: Yes. Long contexts are compute-bound, short ones can be memory-bound. But since NanoFlow batches inputs, it tends to stay compute-bound on average.

Q: What happens when failures occur during NanoFlow's overlapped scheduling?

A: Recovery becomes harder since multiple layers can overlap. For real-world inference systems, it is usually easier to just restart failed queries rather than doing fine-grained fault recovery. There's limited work on this - some early pipeline-parallel fault-tolerance papers by Anna Klimovic's group at ETH.

Q: Why was 1,024 tokens chosen for batch requests in the evaluation?

A: It's a benchmark convention. The token lengths vary by dataset, and 1,024 provides a consistent reference point.

Q: Did they compare Mooncake without disaggregation to maintain fairness?

A: No. They only show results with disaggregated setups.

Q: How are decode and pre-fill instances scheduled?

A: They're both selected ahead of time based on load predictions rather than dynamically reassigned. The selection is lightweight, so parallelizing adds little benefit.

Q: How does scaling affect scheduling overhead?

A: It scales efficiently up to a point; beyond that, block- or cell-level grouping may work better.

Q: How does the prefix-matching cache work?

A: It uses longest-prefix matching on cached keys (K, V pairs) derived from previous tokens to accelerate inference. Local caches only reuse about half of the possible prefixes, while global caches approach optimal reuse, but returns taper once reuse saturates.

Q: When workload characteristics change, do we need to recompile?

A: Yes, recompilation is needed to maintain optimal performance.

Q: How do they decide what's worth caching and how is eviction handled?

A: They cache everything by default to avoid extra computation. Eviction uses LRU.